Pmod OLEDrgb per SPI am Arty A7-35T

Bosch-Logo, HHN-Logo, rotierender Anker und Pong - Dokumentation für Einsteiger (Projekt prj_spi)

Die fünf Bildschirme (aus den echten ROM-Daten gerendert)



sw0	sw1	sw2	sw3	kein / mehrere
Bosch-Logo	Bosch, Anker dreht sich	HHN-Logo	Pong 1 gegen 1	Wähle einen Schalter

Das echte Display ist 0,8 x 0,5 Zoll gross (Digilent) - Farben wirken dort kräftiger als am Bildschirm.

Projekt: prj_spi (FPGA-Praktikum, Arty A7-35T)

Stand: 23.10.2026

Verifikation: GHDL-Simulation bestanden; Betrieb auf dem Board vom Projektteam bestätigt

Autor / Gruppe: noch zu ergänzen

Inhaltsverzeichnis

Auf einen Blick 4
Stand des Projekts 4
In fünf Schritten starten 4
1 Grundlagen: Was ist ein FPGA? 5
1.1 Digital heißt: nur 0 und 1 5
1.2 FPGA und Mikrocontroller im Vergleich 5
1.3 Die zwei wichtigsten Bausteine: LUT und Flip-Flop 5
1.4 Der Takt: alles geschieht gleichzeitig 6
1.5 VHDL: eine Schaltung beschreiben 6
1.6 Zustandsautomaten (FSM) 6
1.7 Pipeline, Zähler, Schieberegister 7
1.8 Vom Text zum laufenden FPGA 7
1.9 Fünf Regeln, die jeder Anfänger kennen sollte 7
2 Die Hardware 8
2.1 Das Board: Arty A7-35T 8
2.2 Die Pmod-Buchse JD 8
2.3 Das Display: Pmod OLEDrgb 8
2.4 Anschluss und Prüfung: GND mit GND, VCC mit VCC 8
2.5 Schalter, Taster, LEDs auf dem Board 9
3 Der Gesamtaufbau 10
3.1 Die Module 10
3.2 Das Top-Modul im Detail 10
4 SPI: wie Daten zum Display kommen 12
4.1 Die Idee 12
4.2 Mode 3: wann ist ein Bit gültig? 12
4.3 Der SPI-Sender spi_tx 13
4.4 Zahlen 13
5 Das Display ansteuern 15
5.1 Der Controller SSD1331 15
5.2 Die Einschaltreihenfolge 15
5.3 Die Konfigurations-Bytes 15
5.4 Die Ablaufsteuerung oled_ctrl 16
5.5 Wie ein Bild übertragen wird 17
5.6 Die Farbe eines Pixels: RGB565 18
5.7 Zeitablauf eines Bildes 18
6 Wie ein Bild entsteht 19
6.1 Pixel für Pixel: "Welche Farbe hat (x, y)?" 19

6.2 Welches Bild? Die Moduswahl ... 19
6.3 Die Bilder im ROM ... 19
6.4 Wie die Bilder entstehen: gen_rom.py ... 20
6.5 Das drehende Bosch-Symbol: Sprites ... 20
6.6 Die Pixel-Pipeline ... 21
6.7 Die Bildschirme im Überblick ... 22
7 Pong ... 23
7.1 Regeln und Bedienung ... 23
7.2 Das Spielfeld ... 23
7.3 Der Spielzustand ... 23
7.4 Was bei jedem Bild passiert ... 24
7.5 Der Abprallwinkel ... 25
7.6 Zufall im FPGA: das LFSR ... 25
7.7 Wie Pong gezeichnet wird ... 25
7.8 Parameter verändern ... 26
8 Eingänge und Reset ... 27
9 Simulation und Tests ... 28
10 Bauen, Programmieren, Fehlersuche ... 29
10.1 Vivado-Projekt und Bitstream ... 29
10.2 Die Pin-Zuordnung (XDC-Datei) ... 29
10.3 Fehlersuche ... 29
11 Erweiterungen und Übungen ... 30
12 Glossar ... 31
13 Quellen ... 32

Auf einen Blick

Dieses Projekt steuert ein **Pmod OLEDrgb** (kleines Farbdisplay, 96 x 64 Pixel) mit einem **FPGA** an. Das Display hängt an der Pmod-Buchse **JD** des **Arty A7-35T**. Je nach Schalterstellung zeigt es ein anderes Bild. Alles - vom Bild bis zum Spiel - ist als Schaltung in **VHDL** beschrieben; es läuft kein Prozessor und kein Programm.

Schalter oben	Anzeige
nur **sw0**	Bosch-Logo (Ring mit Anker, roter Schriftzug BOSCH)
nur **sw1**	Bosch-Logo, der Anker im Ring **dreht sich**
nur **sw2**	HHN-Logo (Hochschule Heilbronn)
nur **sw3**	**Pong** 1 gegen 1 mit den vier Tastern
kein oder mehrere Schalter	Hinweis "Wähle einen Schalter"



Abbildung 1: Die fünf Bildschirme, gerendert aus den echten ROM-Daten des Projekts

Stand des Projekts

- **Simulation (GHDL):** alle Tests bestanden - Initialisierung, Bildformat, alle fünf Bilder pixelgenau, Drehung des Ankers, Pong-Schläger.
- **Hardware:** Das Design läuft laut Projektteam auf dem Arty A7. Vivado-Build und Timing-Berichte sind nicht Teil dieser Dokumentation.
- **Dokumentierter Code:** `src/*.vhd`, `sim/tb_spi_top.vhd`, `scripts/gen_rom.py`, `constraints/arty_a7_35_oled.xdc`.

In fünf Schritten starten

1. Pmod OLEDrgb **bei ausgeschaltetem Board** bis zum Anschlag in **JD** stecken (Abschnitt 2.4).
2. `scripts/create_project.tcl` in Vivado ausführen (erzeugt das Projekt `spi_oled`).
3. In Vivado **Generate Bitstream** und danach im Hardware Manager **Program Device** (USB-Kabel).
4. Einen der vier Schalter nach oben schieben, die anderen unten lassen.
5. Bei Pong: **btn0/btn1** bewegen den linken, **btn2/btn3** den rechten Schläger.

1 Grundlagen: Was ist ein FPGA?

Dieses Kapitel erklärt alles, was du brauchst, um den Rest zu verstehen - auch wenn du noch nie einen FPGA programmiert hast.

1.1 Digital heißt: nur 0 und 1

In digitaler Elektronik hat jede Leitung nur zwei Zustände: **0** (Spannung nahe 0 V) oder **1** (beim Arty A7 etwa 3,3 V). Alles Komplizierte - Zahlen, Bilder, Text - wird aus vielen solcher Leitungen zusammengesetzt. Ein Farbpixel zum Beispiel besteht in diesem Projekt aus 16 Leitungen (16 Bit).

1.2 FPGA und Mikrocontroller im Vergleich

Ein **Mikrocontroller** (z. B. Arduino) hat eine feste Schaltung (den Prozessor), der man ein Programm gibt. Es wird Befehl für Befehl nacheinander abgearbeitet. Ein **FPGA** (Field Programmable Gate Array) ist dagegen ein Chip voller kleiner, frei verschaltbarer Bausteine. Man beschreibt, **wie sie verbunden sein sollen**; danach existiert diese Schaltung wirklich und alle Teile arbeiten **gleichzeitig**.

	Mikrocontroller	FPGA
Was man schreibt	Programm (C, Python ...)	Schaltungsbeschreibung (VHDL, Verilog)
Ablauf	Befehle nacheinander	alle Bausteine gleichzeitig, im Takt
Zeitverhalten	schwankt (Interrupts, Compiler)	exakt: jeder Schritt = 1 Taktperiode (hier 10 ns)
Gut für	Steuerungen, Software	schnelle parallele Signalverarbeitung, eigene Schnittstellen
Beispiel hier	-	SPI-Sender, Bildgenerator und Spiel laufen parallel

1.3 Die zwei wichtigsten Bausteine: LUT und Flip-Flop

LUT (Look-Up Table): ein winziger Speicher, der jede Logikfunktion kann. Die Eingänge sind die Adresse, im Speicher steht das Ergebnis. Eine UND-Verknüpfung ist zum Beispiel eine Tabelle mit vier Zeilen. Ein Artix-7 hat 6-Eingangs-LUTs; der Arty A7-35T enthält laut Digilent 5 200 Slices zu je vier LUTs und acht Flip-Flops [1].

Flip-Flop (FF): speichert **ein Bit**. Es übernimmt seinen Eingang D nur bei der **steigenden Taktflanke** und hält den Wert danach bis zur nächsten Flanke. Mehrere Flip-Flops nebeneinander nennt man **Register**.

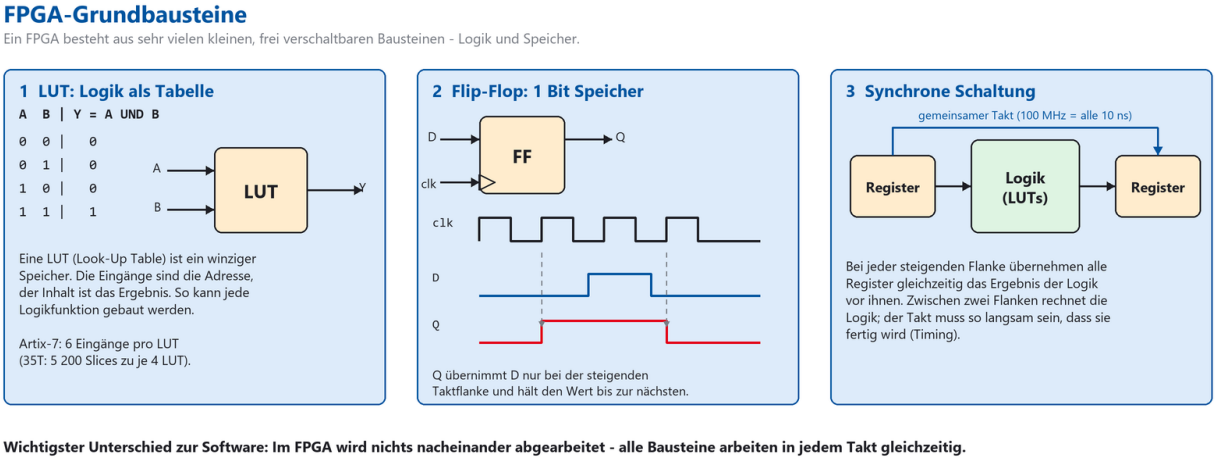


Abbildung 2: LUT, Flip-Flop und synchrone Schaltung

1.4 Der Takt: alles geschieht gleichzeitig

Der Arty A7 hat einen Quarzoszillator mit **100 MHz** (Pin E3) [1]. Eine Periode dauert also **10 ns**. Bei jeder steigenden Flanke übernehmen alle Flip-Flops im ganzen Chip gleichzeitig das Ergebnis der Logik, die vor ihnen liegt. Zwischen zwei Flanken rechnet die Logik; sie muss in 10 ns fertig werden, sonst gibt es Timing-Fehler. Diese Bauweise heißt **synchron** und ist die wichtigste Regel im FPGA-Entwurf.

> **Hinweis:** Merksatz: **Register - Logik - Register.** Jede Berechnung geht von einem Register zum nächsten. Je kürzer die Logik dazwischen, desto schneller darf der Takt sein. Deshalb gibt es in diesem Projekt eine **Pipeline** (Abschnitt 6.6): sie zerlegt einen langen Rechenweg in kurze Stücke.

1.5 VHDL: eine Schaltung beschreiben

VHDL ist eine **Beschreibungssprache**, keine Programmiersprache. Eine Zeile bedeutet nicht "tu das jetzt", sondern "hier gibt es diese Schaltung". Jedes Modul hat zwei Teile:

- **entity** - die Schnittstelle (Name, Ein- und Ausgänge, wie die Anschlüsse eines Bausteins).
- **architecture** - der Inhalt (Signale, Prozesse, Verbindungen).

Ein **process (clk)** mit `if rising_edge(clk)` beschreibt Register: Alles darin wird bei jeder steigenden Taktflanke ausgeführt. Als Beispiel der Entprell-Filter des Projekts (Modul `input_filter`):

```
process (clk)
begin
    if rising_edge(clk) then
        sync <= sync(0) & d;               -- 2-Flip-Flop-Kette (Eingang einsynchronisieren)
        if rst = '1' then
            stable <= '0';
            cnt    <= 0;
        elsif sync(1) = stable then        -- nichts hat sich geändert
            cnt <= 0;
        elsif cnt = DEBOUNCE_CYCLES then   -- 10 ms lang anders -> übernehmen
            cnt    <= 0;
            stable <= sync(1);
        else
            cnt <= cnt + 1;                -- zählen, wie lange der neue Pegel anliegt
        end if;
    end if;
end process;
```

VHDL	Bedeutung
`<=`	Signalzuweisung: der neue Wert gilt **ab der nächsten Taktflanke**
`:=`	Variable: sofortige Zuweisung nur innerhalb eines Prozesses (Rechenzwischenwerte)
`&`	Zusammenhängen von Bitfolgen: `sync(0) & d` schiebt d hinten an
`x"AE"`	Hexadezimalzahl, hier 1010 1110
`(others => '0')`	alle Bits auf 0
`7 downto 0`	ein 8-Bit-Vektor, Bit 7 ist das höchstwertige (links)
`std_logic, unsigned`	Bit bzw. vorzeichenlose Zahl als Bitvektor
`generic`	Parameter eines Moduls (z. B. Taktfrequenz, HALF beim SPI)

1.6 Zustandsautomaten (FSM)

Viele Abläufe bestehen aus Schritten, die nacheinander kommen - wie eine **Ampel**: rot, rot-gelb, grün, gelb. Ein Zustandsautomat (Finite State Machine) merkt sich in einem Register, in welchem Schritt er ist (`state`), und entscheidet bei jedem Takt, ob er bleibt oder weitergeht. In VHDL schreibt man das als `case state is ...`.

Dieses Projekt hat drei Automaten: den SPI-Sender (`spi_tx`), die Display-Steuerung (`oled_ctrl`) und - als Spielablauf - Pong.

1.7 Pipeline, Zähler, Schieberegister

- **Zähler:** ein Register, das bei jedem Takt um 1 größer wird. Damit misst man Zeit ("warte 20 ms" = 2 000 000 Takte zählen).
- **Schieberegister:** eine Kette von Flip-Flops, in der die Bits pro Takt eine Stelle weiterrutschen. Der SPI-Sender schiebt so ein Byte Bit für Bit nach draußen.
- **Pipeline:** mehrere Register hintereinander mit wenig Logik dazwischen. Jede Stufe erledigt einen Teil; dadurch ist der Takt hoch möglich.

1.8 Vom Text zum laufenden FPGA

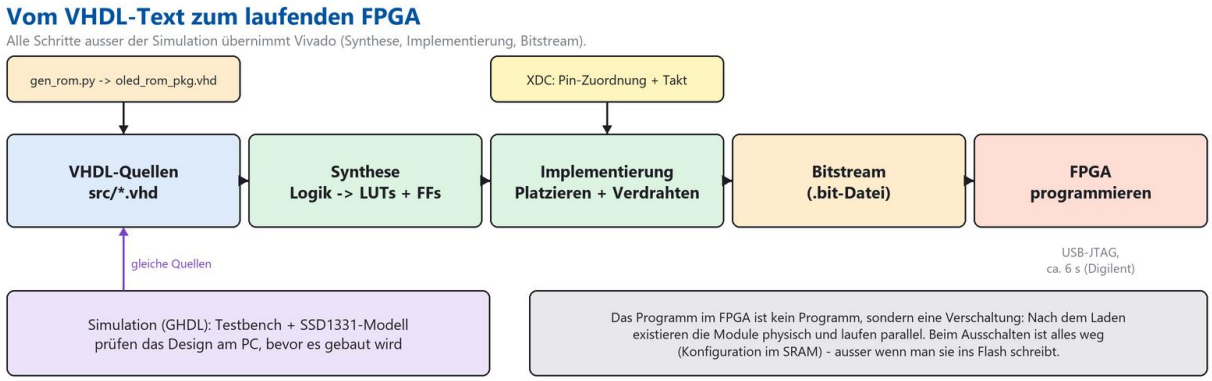


Abbildung 3: Werkzeugkette: Quellen, Simulation, Synthese, Implementierung, Bitstream

- **Simulation:** das Design läuft am PC; eine **Testbench** legt Eingangssignale an und prüft die Ausgänge mit `assert`. Hier mit **GHDL**. Fehler werden so gefunden, ohne dass Hardware nötig ist.
- **Synthese:** Vivado macht aus der Beschreibung LUTs und Flip-Flops.
- **Implementierung:** Vivado legt fest, welche LUT wo sitzt und wie sie verdrahtet werden, und weist den Pins die Namen aus der **XDC-Datei** zu.
- **Bitstream:** die fertige Konfigurationsdatei (`.bit`). Beim Arty A7-35T ist sie rund 17,5 Mbit groß; das Laden über USB-JTAG dauert laut Digilent etwa 6 Sekunden [1].
- **Flüchtig:** Der FPGA vergisst die Schaltung beim Ausschalten (SRAM-Konfiguration). Soll sie bleiben, schreibt man sie in den Flash [1].

1.9 Fünf Regeln, die jeder Anfänger kennen sollte

6. **Alles Getaktete hängt an einem Takt.** Hier: `CLK100MHZ`.
7. **Eingänge von außen immer einsynchronisieren** (2 Flip-Flops), sonst können Zufallswerte entstehen (Metastabilität).
8. **Mechanische Taster prellen** - mit einem Filter entprellen (Kapitel 8).
9. **Zeit = Takte zählen.** Es gibt kein `sleep()`.
10. **Erst simulieren, dann bauen.** Ein Fehler im Simulator ist in Sekunden gefunden, auf der Hardware oft nie.

2 Die Hardware

2.1 Das Board: Arty A7-35T

Merkmal	Wert (Quelle [1])
FPGA	Xilinx Artix-7 **XC7A35TICSG324-1L**
Takt	ein 100-MHz-Quarzoszillator an Pin **E3**
Bedienelemente	4 Schiebeschalter, 4 Taster, 1 RESET-Taster, 4 LEDs, 4 RGB-LEDs
Erweiterung	4 Pmod-Buchsen (JA, JB, JC, JD), Arduino-Shield-Anschluss
Programmieren	USB-JTAG (Micro-USB), Quad-SPI-Flash 16 MB
Spannungen	Pmod-Pins arbeiten mit 3,3 V; nie über 3,4 V treiben

2.2 Die Pmod-Buchse JD

Ein Pmod-Anschluss ist eine Buchse mit 2 x 6 Kontakten im Abstand von 2,54 mm. Von den zwölf Kontakten sind **zwei Masse** (Pin 5 und 11), **zwei 3,3-V-Versorgung** (Pin 6 und 12) und **acht Signale** [1]. Die VCC- und GND-Pins können laut Digilent bis zu 1 A liefern, solange das Board insgesamt nicht überlastet wird [1].

JD ist ein **Standard-Pmod**: Die Signalleitungen laufen über **200-Ohm-Schutzwiderstände**. Das schützt vor Kurzschlüssen und begrenzt die maximale Schaltgeschwindigkeit - für SPI mit 6,25 MHz ist das unkritisch. JB und JC sind dagegen High-Speed-Pmods ohne Schutzwiderstände [1].

2.3 Das Display: Pmod OLEDrgb

- Anzeige mit **96 x 64 Pixeln**, jeder Pixel mit **16 Bit Farbe** (RGB 5-6-5) [2].
- Steuerchip: **SSD1331** von Solomon Systech [2].
- Schnittstelle: **SPI** (nur Schreiben, es kommt nichts zurück) [2].
- Versorgung 2,7 ... 3,5 V, empfohlen 3,3 V [2].
- Stromaufnahme bis ca. **75,5 mA**, wenn alle Pixel weiß und maximal hell sind [2] - deshalb bleibt das Display im Projekt moderat eingestellt.
- Zwei Leitungen schalten die Versorgung: **PMODEN** (Logik) und **VCCEN** (hohe Panelspannung). Sind beide aus, braucht das Modul etwa 200 nA [2].

2.4 Anschluss und Prüfung: GND mit GND, VCC mit VCC

Das Modul passt **1:1** auf JD: Pin 1 des Moduls ist Pin 1 von JD usw. Dabei liegen GND-Pins auf GND und VCC-Pins auf VCC automatisch richtig, **wenn der Stecker direkt und nicht versetzt eingesteckt ist**.

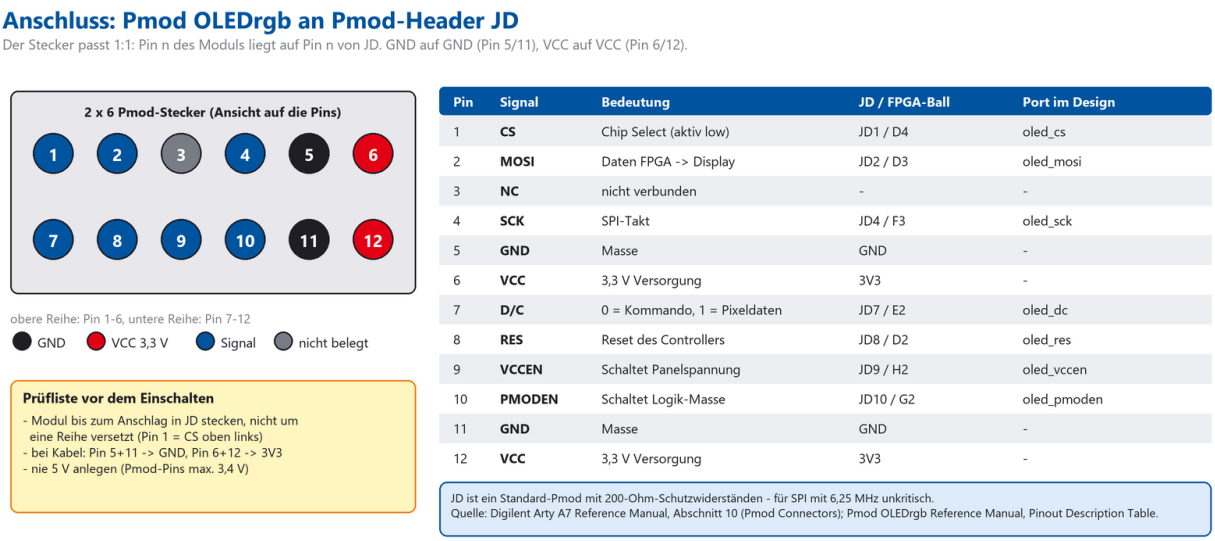


Abbildung 4: Pin-Belegung des Pmod OLEDrgb und Zuordnung zu JD / Port-Namen

So prüfst du den Anschluss

11. **Sichtprüfung:** Das Modul sitzt mit allen 12 Kontakten in der Buchse, nicht um eine Reihe oder um Pins verschoben. Obere Reihe = Pin 1 bis 6, untere Reihe = Pin 7 bis 12.
12. **Bei Kabelverbindung:** Pin 5 **und** 11 des Moduls auf GND, Pin 6 **und** 12 auf 3V3 (nicht umgekehrt!). Die Signalpins CS, MOSI, SCK, D/C, RES, VCCEN, PMODEN sind nicht austauschbar, weil die XDC-Datei sie auf feste FPGA-Pins legt. Pin 3 bleibt frei.
13. **Messen (Board an, Modul gesteckt):** Zwischen Pin 6 und Pin 5 des Moduls müssen etwa **3,3 V** liegen. Mit dem Durchgangsprüfer (Board aus): Pin 5 und Pin 11 sind miteinander verbunden, Pin 6 und Pin 12 ebenfalls, und zwischen Pin 5 und Pin 6 ist **kein** Durchgang (sonst Kurzschluss).
14. **Nie 5 V anlegen.** Die Pmod-Pins vertragen höchstens 3,4 V [1].

Achtung: Stecke Module nur bei **ausgeschaltetem Board** um. Falsch herum gesteckte Versorgungspins können das Display zerstören.

2.5 Schalter, Taster, LEDs auf dem Board

Bauteil	Port im Design	FPGA-Pin	Verhalten (nach [1])
Takt	CLK100MHZ	E3	100 MHz
RESET-Taster	ck_rst	C2	**aktiv low**: im Ruhezustand 1, gedrückt 0
sw0 ... sw3	sw[0] ... sw[3]	A8, C11, C10, A10	Schiebeschalter: oben = 1, unten = 0
btn0 ... btn3	btn[0] ... btn[3]	D9, C9, B9, B8	Taster: Ruhezustand 0, gedrückt 1
LED0 ... LED3	led[0] ... led[3]	H5, J5, T9, T10	1 = LED an (spiegeln die Schalter)

3 Der Gesamtaufbau

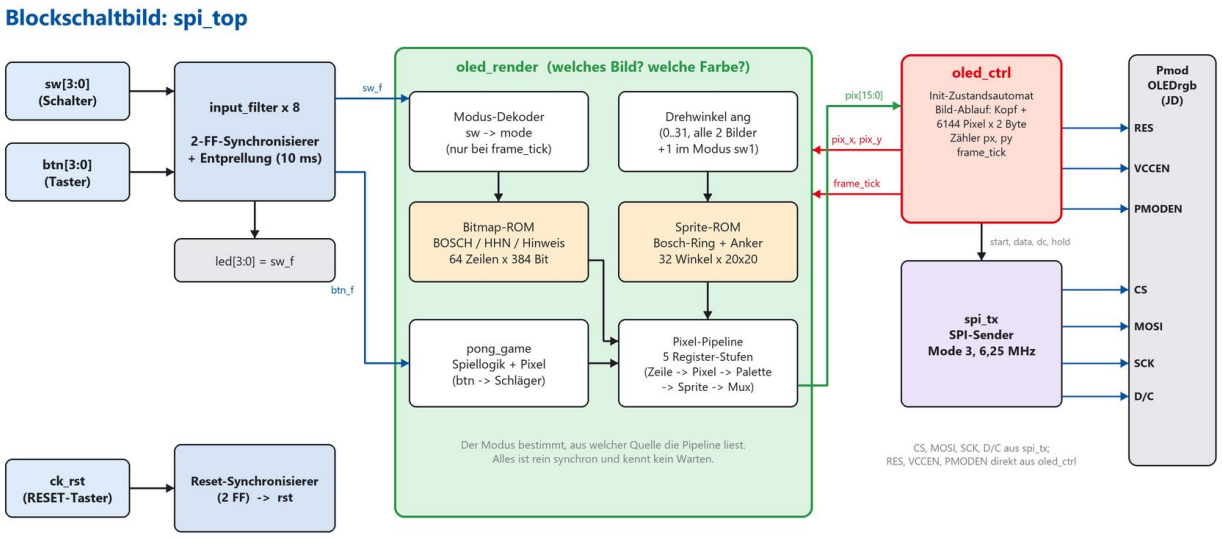


Abbildung 5: Blockschaltbild des Top-Moduls spi_top

Der Datenfluss in einem Satz: **Die Schalter wählen das Bild (`oled_render`), `oled_ctrl` fragt es Pixel für Pixel ab und schickt jedes Pixel über `spi_tx` an das Display.**

3.1 Die Module

Modul (Datei)	Aufgabe
`spi_top`	Top-Modul: verbindet alles, synchronisiert den Reset, erzeugt acht Entprell-Filter, spiegelt die Schalter auf die LEDs
`input_filter`	2-Flip-Flop-Synchronisierer + Entprellung (10 ms) für Schalter und Taster
`oled_render`	Pixelgenerator: wählt anhand der Schalter das Bild und liefert zu (x, y) eine 16-Bit-Farbe
`pong_game`	Spiellogik und Zeichnen von Pong
`oled_rom_pkg`	**Erzeugt** von `gen_rom.py`: Bitmaps (Bosch, HHN, Hinweis), Paletten, 32 Anker-Sprites
`oled_ctrl`	Ablaufsteuerung: Einschalten, Initialisieren, endlos Bilder senden; zählt x/y
`spi_tx`	SPI-Sender: schickt ein Byte seriell an das Display
`tb_spi_top`	Testbench mit Display-Modell

3.2 Das Top-Modul im Detail

Im Top-Modul stecken drei Dinge. **Erstens** der Reset: Der RESET-Taster ist aktiv low, er wird invertiert und durch zwei Flip-Flops geführt (`rst_sync`), damit er mit dem Takt synchron ist. **Zweitens** acht `input_filter` (vier Schalter, vier Taster) per `generate`-Schleife. **Drittens** die Verbindung der Module: `oled_render` liefert `pix` an `oled_ctrl`, und `oled_ctrl` liefert `pix_x`, `pix_y` und `frame_tick` zurück.

Parameter (generic)	Standard	Bedeutung
`CLK_HZ`	100 000 000	Taktfrequenz; daraus folgen die 10 ms Entprellzeit und die Mikrosekunden-Zeiten
`SPI_HALF`	8	Takte pro halber SCK-Periode: SCK = 100 MHz / (2 x 8) = 6,25 MHz

PIPE_WAIT (oled_ctrl)	12	Takte Wartezeit, bis die Pipeline das Pixel berechnet hat
ROT_FRAMES (oled_render)	2	Bilder pro Drehschritt des Ankers

4 SPI: wie Daten zum Display kommen

4.1 Die Idee

SPI (Serial Peripheral Interface) ist eine einfache serielle Schnittstelle: Statt 8 Leitungen für ein Byte nutzt man **eine** Datenleitung und schickt die Bits nacheinander. Ein Taktsignal sagt, wann ein Bit gültig ist. Unser Display ist ein **Slave**; der FPGA ist der **Master** und bestimmt den Takt.

Leitung	Richtung	Bedeutung
SCK	FPGA -> Display	Takt: jede Flanke gehört zu einem Bit
MOSI	FPGA -> Display	Master Out, Slave In: die Datenbits
CS (CS_N)	FPGA -> Display	Chip Select, **aktiv low**: nur wenn 0, hört das Display zu
D/C	FPGA -> Display	Zusatzleitung des SSD1331: 0 = Byte ist ein **Kommando**, 1 = Byte ist **Pixeldaten**
(MISO)	-	gibt es nicht: das Display kann nur empfangen [2]

4.2 Mode 3: wann ist ein Bit gültig?

Das Display arbeitet in **SPI-Mode 3**: Der Takt SCK **ruht auf 1**. Der Sender legt ein neues Bit bei der **fallenden** Flanke auf MOSI; das Display liest es bei der **steigenden** Flanke. Die kürzeste erlaubte Taktperiode ist **150 ns** [2]; das Projekt verwendet 160 ns (6,25 MHz).

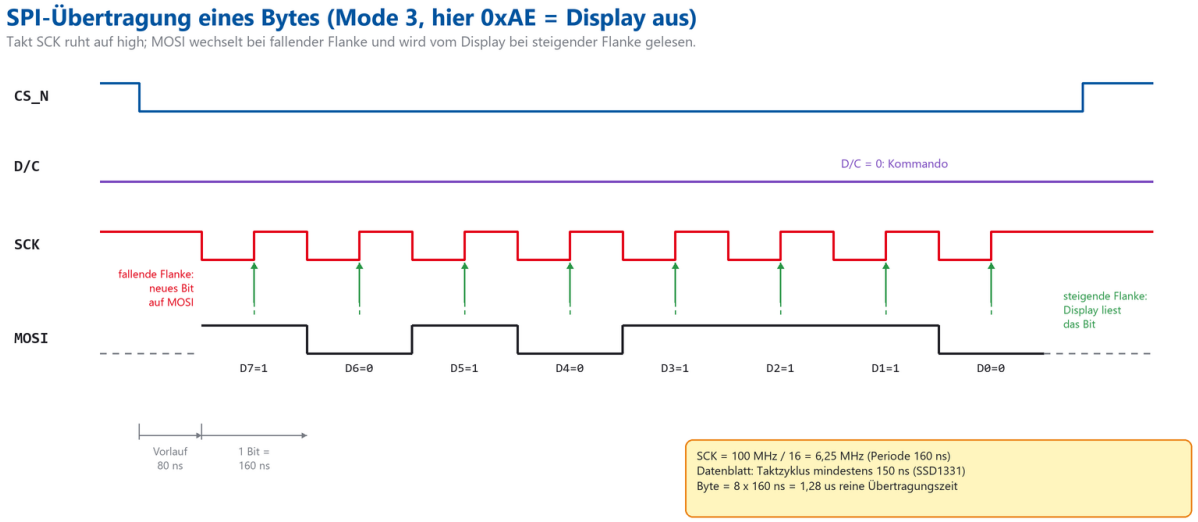


Abbildung 6: Ein Byte (0xAE) über SPI Mode 3

Ablauf für das Byte **0xAE = 1010 1110**: CS geht auf 0. Nach einem kurzen Vorlauf (80 ns) fällt SCK, und das erste Bit (1) liegt an MOSI. Nach 80 ns steigt SCK - das Display übernimmt die 1. Nach weiteren 80 ns fällt SCK wieder, das nächste Bit (0) wird angelegt, und so weiter. Nach acht Takten ist das Byte übertragen; CS geht wieder auf 1 (bei Pixeldaten bleibt es auf 0).

4.3 Der SPI-Sender spi_tx

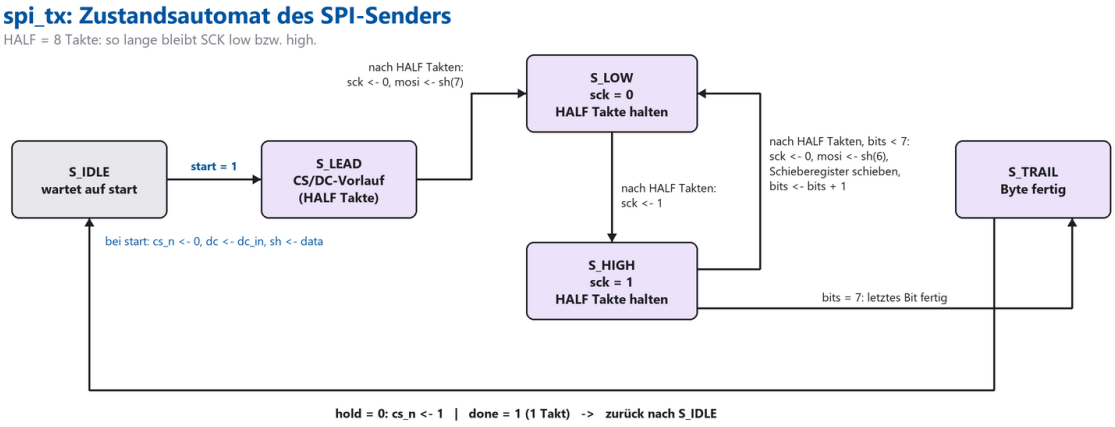


Abbildung 7: Zustandsautomat von spi_tx

Zustand	Was passiert
S_IDLE	wartet. Kommt `start = 1`, werden `cs_n <- 0`, `dc <- dc_in`, das Schieberegister `sh <- data` gesetzt
S_LEAD	wartet HALF Takte (CS und D/C stabil), dann `sck <- 0` und das erste Bit `mosi <- sh(7)`
S_LOW	SCK bleibt HALF Takte auf 0, dann `sck <- 1` (steigende Flanke)
S_HIGH	SCK bleibt HALF Takte auf 1; danach entweder nächstes Bit (`sck <- 0`, `mosi <- sh(6)`, schieben) oder - beim 8. Bit - weiter zu S_TRAIL
S_TRAIL	Ist `hold = 0`, geht CS auf 1. done meldet **einen Takt lang** "Byte fertig"

```
when S_LOW =>
    if tmr = HALF - 1 then
        tmr   <= 0;
        sck   <= '1';                -- steigende Flanke: Display liest das Bit
        state <= S_HIGH;
    else
        tmr <= tmr + 1;
    end if;

when S_HIGH =>
    if tmr = HALF - 1 then
        tmr <= 0;
        if bits = 7 then
            state <= S_TRAIL;        -- alle 8 Bits gesendet
        else
            sck   <= '0';            -- fallende Flanke: nächstes Bit anlegen
            mosi  <= sh(6);
            sh    <= sh(6 downto 0) & '0';
            bits  <= bits + 1;
            state <= S_LOW;
        end if;
    else
        tmr <= tmr + 1;
    end if;
```

Der Eingang `hold` ist wichtig für die Geschwindigkeit: Kommandos werden mit `hold = 0` gesendet (CS geht nach jedem Byte hoch). Beim Pixelstrom ist `hold = 1`, CS bleibt für das ganze Bild auf 0 - bis auf das allerletzte Byte.

4.4 Zahlen

Größe	Wert
Takt des FPGA	100 MHz (10 ns)

SCK	100 MHz / 16 = **6,25 MHz** (160 ns Periode; erlaubt: min. 150 ns [2])
Übertragung eines Bytes	8 x 160 ns = 1,28 µs
Ein Byte inklusive Vorlauf und Handshake	ca. 140 Takte = 1,4 µs
Ein Pixel (2 Byte + 12 Takte Wartezeit)	ca. 294 Takte = 2,94 µs

5 Das Display ansteuern

5.1 Der Controller SSD1331

Hinter dem Glas sitzt der Chip SSD1331. Er hat einen eigenen **Bildspeicher** (96 x 64 Pixel). Was dort steht, zeigt er dauerhaft an - der FPGA muss das Bild nicht ständig wiederholen, sondern nur dann schreiben, wenn sich etwas ändert. In diesem Projekt wird trotzdem **immer wieder das ganze Bild** geschrieben, weil Pong und die Drehung ständig neue Bilder brauchen.

Der Chip versteht **Kommandos** (D/C = 0), gefolgt von Parameter-Bytes, und nimmt **Pixeldaten** (D/C = 1) entgegen.

5.2 Die Einschaltreihenfolge

OLED-Displays brauchen eine feste Reihenfolge beim Einschalten; die hohe Panelspannung darf erst fließen, wenn der Controller konfiguriert ist. Die Reihenfolge stammt aus dem Digilent-Handbuch [2].

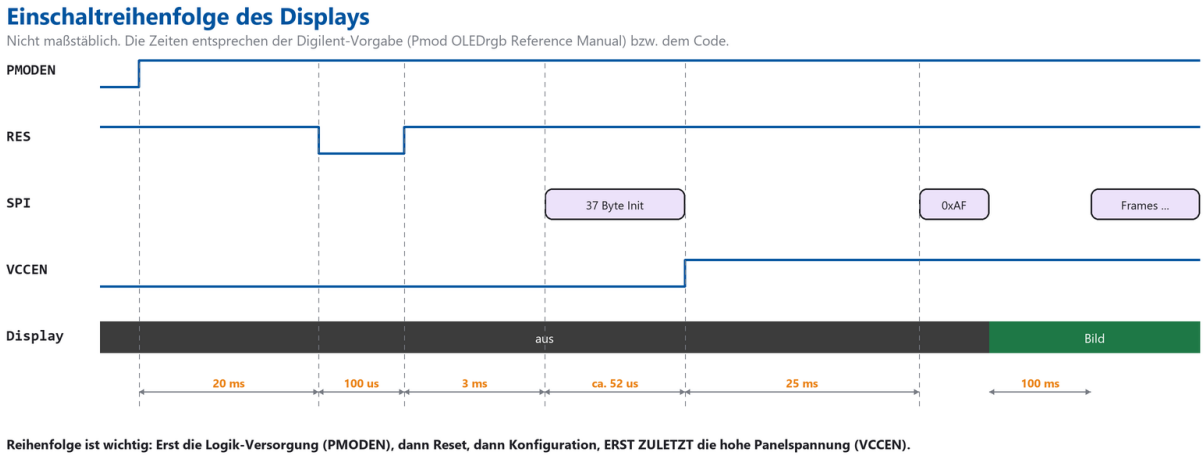


Abbildung 8: Einschaltreihenfolge (nicht maßstäblich)

Schritt	Aktion	Zeit
1	PMODEN = 1 (Logik-Versorgung des Moduls ein)	20 ms warten, bis die Spannung stabil ist
2	RES = 0, dann RES = 1 (Controller zurücksetzen)	RES mind. 3 µs low (Code: 100 µs), danach 3 ms warten
3	37 Kommando-Bytes (Konfiguration), D/C = 0	ca. 52 µs
4	VCCEN = 1 (Panelspannung ein)	25 ms warten
5	Kommando **0xAF** (Display ein)	100 ms warten
6	ab jetzt: Bilder senden	dauerhaft

5.3 Die Konfigurations-Bytes

Die 37 Bytes in `INIT_CMD` sind Kommandos mit Parametern. Die Werte entsprechen der Digilent-Vorgabe [2]; Details zu jedem Befehl stehen im SSD1331-Datenblatt (von Digilent verlinkt).

Bytes	Bedeutung
`AE`	Display aus
`A0 72`	Remap und Farbformat: 65k Farben, RGB-Reihenfolge, Ausrichtung des Bildes
`A1 00`	Startzeile = 0

A2 00	Display-Offset = 0
A4	normale Anzeige (nicht invertiert)
A8 3F	Multiplex-Verhältnis 64: alle 64 Zeilen nutzen
AD 8E	Master-Konfiguration: externe Panelspannung (VCC) erforderlich
B0 0B	Energiesparmodus aus
B1 31	Phasenlängen (Lade/Entladedauer eines Pixels)
B3 F0	Takteiler und interne Oszillatorfrequenz
8A 64, 8B 78, 8C 64	zweite Vorladegeschwindigkeit für die Farben A (rot), B (grün), C (blau)
BB 3A	Vorlade-Spannung (ca. 45 % von VCC)
BE 3E	VCOMH-Pegel (ca. 83 % von VCC)
87 06	Master-Strom (Referenzstrom der Treiber)
81 91, 82 50, 83 7D	Kontrast (Helligkeit) für Farbe A, B, C
2E	Scrollen aus

Hinweis: Unterschied zur Digilent-Referenz: Deren Liste sendet zusätzlich FD 12 (Kommandos entsperren) als erstes und 25 00 00 5F 3F (Bildspeicher löschen) vor VCCEN [2]. Das Projekt lässt diese beiden Schritte weg - das Display läuft laut Projektteam trotzdem, und der Bildspeicher wird sowieso gleich komplett überschrieben. Sollte ein anderes Display-Exemplar nicht starten, sind das die ersten Schritte zum Ausprobieren.

5.4 Die Ablaufsteuerung oled_ctrl

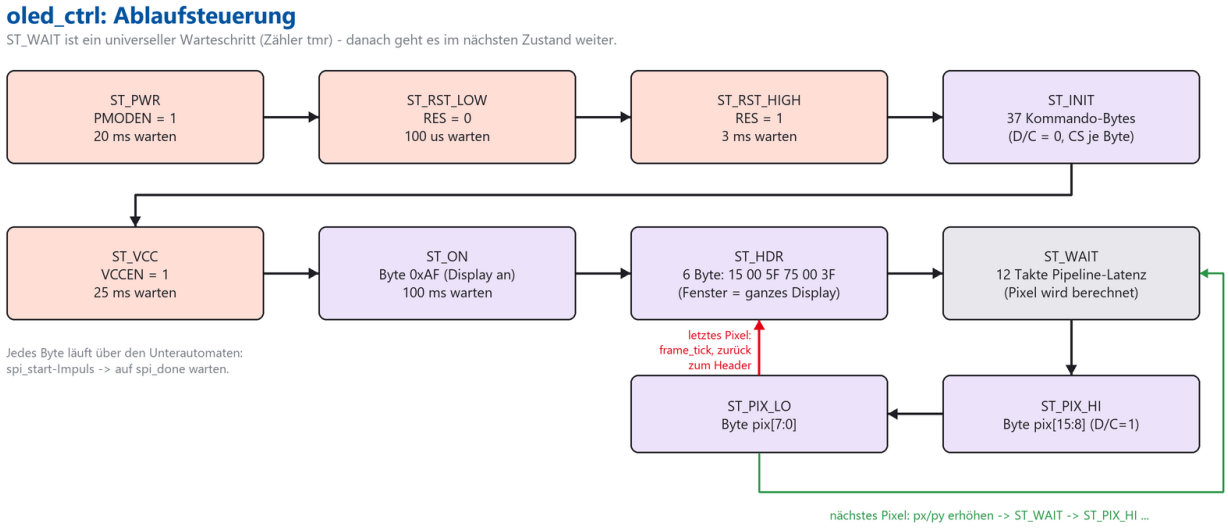


Abbildung 9: Zustandsautomat von oled_ctrl

- **ST_PWR, ST_RST_LOW, ST_RST_HIGH:** Strom ein, Reset-Puls (siehe 5.2).
- **ST_INIT:** die 37 Bytes nacheinander. Für jedes Byte wird `spi_start` einen Takt lang gesetzt, dann wartet der Automat auf `spi_done` (Variable `waiting`).
- **ST_VCC, ST_ON:** Panelspannung, Display-ein-Kommando.
- **ST_HDR:** sechs Bytes 15 00 5F 75 00 3F legen das **Fenster** fest: Spalten 0 bis 95 (0x15, Start 00, Ende 5F) und Zeilen 0 bis 63 (0x75, Start 00, Ende 3F). Danach schreibt jedes Pixel automatisch an die nächste Stelle.
- **ST_WAIT:** wartet `PIPE_WAIT` = 12 Takte, bis die Pipeline das Pixel für (x, y) berechnet hat.

- **ST_PIX_HI / ST_PIX_LO:** schickt das obere und dann das untere Byte des Pixels. Nach dem letzten Pixel (95, 63) wird `frame_tick` einen Takt lang 1 und das nächste Bild beginnt mit dem Kopf.

```
when ST_INIT =>
    if waiting = '0' then              -- Byte anstoßen
        spi_data  <= INIT_CMD(idx);
        spi_dc    <= '0';              -- Kommando
        spi_hold  <= '0';
        spi_start <= '1';
        waiting   <= '1';
    elsif spi_done = '1' then          -- Byte ist fertig
        waiting <= '0';
        if idx = INIT_CMD'high then
            idx   <= 0;
            state <= ST_VCC;           -- alle 37 Bytes gesendet
        else
            idx <= idx + 1;
        end if;
    end if;
```

5.5 Wie ein Bild übertragen wird

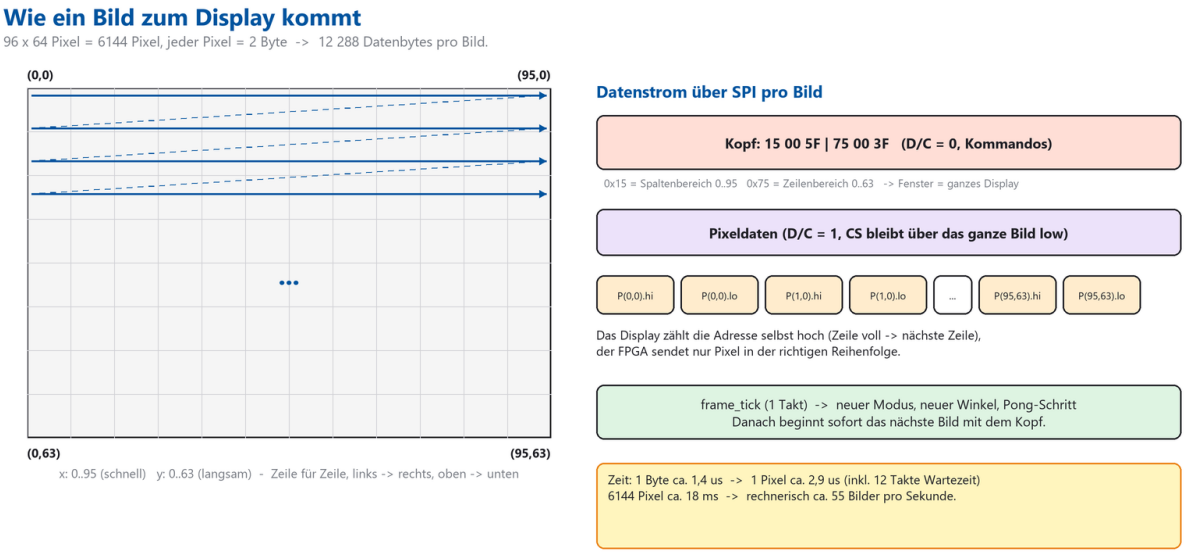


Abbildung 10: Pixelreihenfolge und Datenstrom pro Bild

Ein Bild hat 96 x 64 = **6144 Pixel**, jedes besteht aus 2 Byte, also **12 288 Datenbytes**. Die Pixel werden zeilenweise gesendet, links nach rechts und oben nach unten. Die Zähler `px` (0 bis 95) und `py` (0 bis 63) in `oled_ctrl` zeigen, welches Pixel gerade dran ist; `oled_render` bekommt sie als `pix_x`, `pix_y`.

Das Display zählt die Speicheradresse selbst hoch - der FPGA muss pro Pixel keine Adresse senden. Hat das Display eine Zeile voll, springt es in die nächste.

5.6 Die Farbe eines Pixels: RGB565

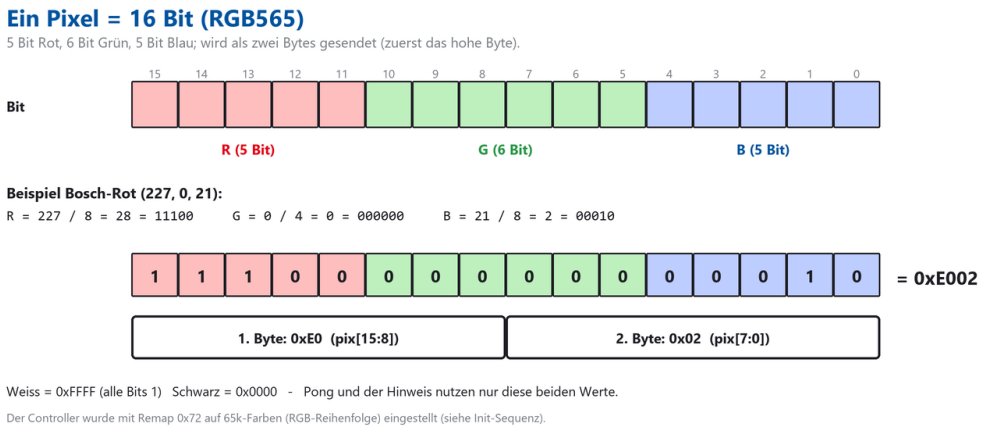


Abbildung 11: 16-Bit-Pixel: 5 Bit Rot, 6 Bit Grün, 5 Bit Blau

Jedes Pixel ist eine 16-Bit-Zahl: oben 5 Bit Rot, in der Mitte 6 Bit Grün, unten 5 Bit Blau. Aus einer 8-Bit-Farbe (0 bis 255) wird die kleinere Zahl durch **Weglassen der niedrigen Bits** (Rot: 227 / 8 = 28). Das Display bekommt zuerst das **obere** Byte (`pix[15:8]`), dann das untere. Bosch-Rot (227, 0, 21) wird so zu `0xE002`, Weiß ist `0xFFFF`, Schwarz `0x0000`.

5.7 Zeitablauf eines Bildes

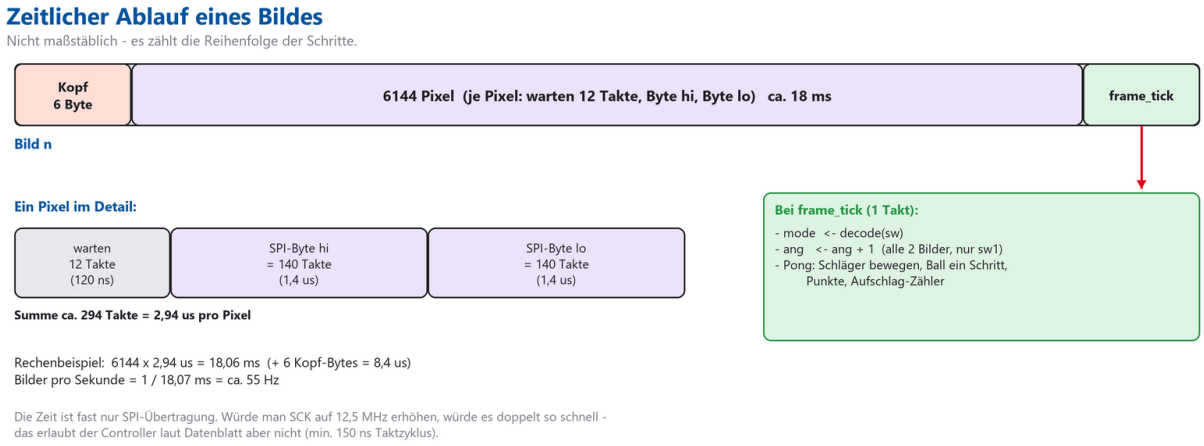


Abbildung 12: Zeitaufteilung eines Bildes

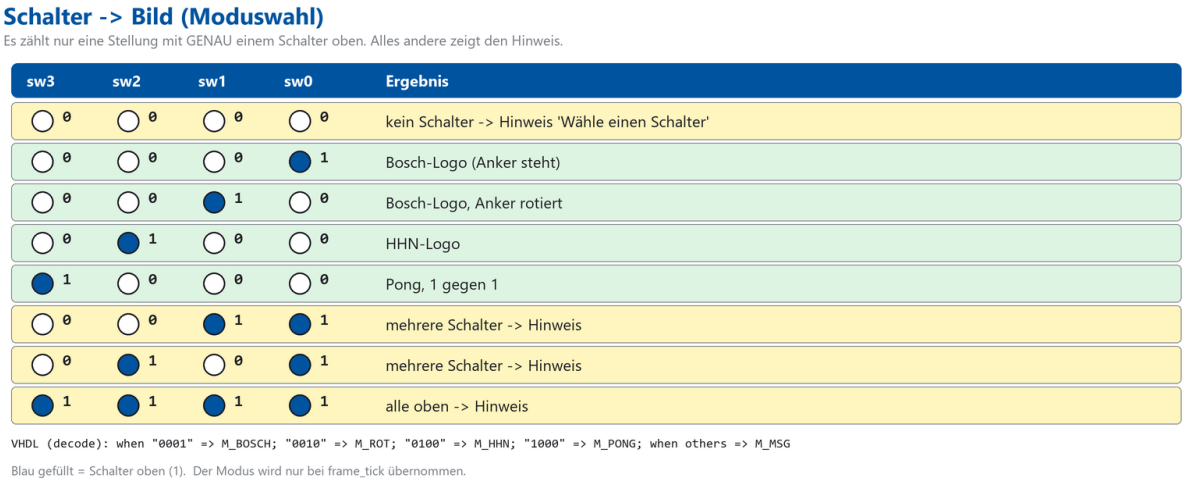
Pro Pixel vergehen ca. 294 Takte: 12 Takte Wartezeit für die Pipeline und je 140 Takte für die beiden SPI-Bytes. 6144 Pixel x 2,94 µs ergeben ca. **18 ms**, das sind **rechnerisch ca. 55 Bilder pro Sekunde** (nicht gemessen). Genau am Bildende - `frame_tick` - werden Modus, Drehwinkel und Spielzustand aktualisiert; dadurch ändert sich innerhalb eines Bildes nie etwas, das Bild zerreißt nicht.

6 Wie ein Bild entsteht

6.1 Pixel für Pixel: "Welche Farbe hat (x, y)?"

Stell dir `oled_render` als Maler vor, der blind arbeitet: Er bekommt nur eine Koordinate (x, y) zugerufen und antwortet mit einer Farbe. Es gibt **keinen Bildspeicher** im FPGA, in dem das Bild als Ganzes liegt. Für jedes der 6144 Pixel wird die Antwort frisch berechnet. Das ist platzsparend, weil nur das nötig ist, was man gerade sendet.

6.2 Welches Bild? Die Moduswahl



Schalter -> Bild (Moduswahl)

Es zählt nur eine Stellung mit GENAU einem Schalter oben. Alles andere zeigt den Hinweis.

sw3	sw2	sw1	sw0	Ergebnis
○ 0	○ 0	○ 0	○ 0	kein Schalter -> Hinweis 'Wähle einen Schalter'
○ 0	○ 0	○ 0	● 1	Bosch-Logo (Anker steht)
○ 0	○ 0	● 1	○ 0	Bosch-Logo, Anker rotiert
○ 0	● 1	○ 0	○ 0	HHN-Logo
● 1	○ 0	○ 0	○ 0	Pong, 1 gegen 1
○ 0	○ 0	● 1	● 1	mehrere Schalter -> Hinweis
○ 0	● 1	○ 0	● 1	mehrere Schalter -> Hinweis
● 1	● 1	● 1	● 1	alle oben -> Hinweis

VHDL (decode): when "0001" => M_BOSCH; "0010" => M_ROT; "0100" => M_HHN; "1000" => M_PONG; when others => M_MSG

Blau gefüllt = Schalter oben (1). Der Modus wird nur bei frame_tick übernommen.

Abbildung 13: Schalterstellung bestimmt das Bild

```
function decode (s : std_logic_vector(3 downto 0)) return mode_t is
begin
    case s is
        when "0001" => return M_BOSCH;
        when "0010" => return M_ROT;
        when "0100" => return M_HHN;
        when "1000" => return M_PONG;
        when others => return M_MSG;      -- keiner oder mehrere Schalter
    end case;
end function;
```

`when others` fängt alle anderen 12 Möglichkeiten ab, also auch "kein Schalter" und "mehrere Schalter". Der Modus wird nur bei `frame_tick` in das Register `mode` übernommen.

6.3 Die Bilder im ROM

Die drei Bilder (Bosch-Schriftzug, HHN-Logo, Hinweistext) liegen als **Konstanten im VHDL-Code**. Ein Pixel ist nicht als Farbe gespeichert, sondern als **Palettenindex** (4 Bit, also 16 mögliche Farben pro Bild). Die Palette hält zu jedem Index die RGB565-Farbe. Das spart Speicher: statt 16 Bit pro Pixel nur 4 Bit.

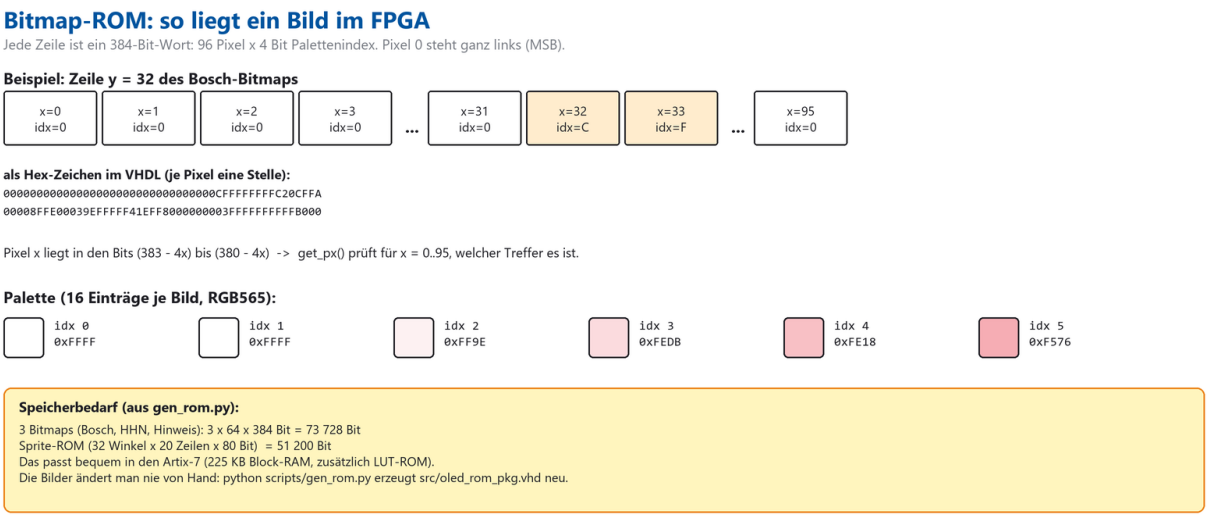


Abbildung 14: Aufbau einer ROM-Zeile und der Palette

```
function get_px (row : std_logic_vector(383 downto 0); xp : unsigned(6 downto 0))
    return std_logic_vector is
    variable r : std_logic_vector(3 downto 0) := "0000";
begin
    for i in 0 to 95 loop
        if to_integer(xp) = i then
            r := row(383 - 4 * i downto 380 - 4 * i);   -- 4 Bit von Pixel i
        end if;
    end loop;
    return r;
end function;
```

Die `for`-Schleife ist **keine Zeitschleife**. Sie beschreibt einen Multiplexer, der aus 96 Möglichkeiten die passende auswählt. Der FPGA baut daraus eine feste Schaltung.

6.4 Wie die Bilder entstehen: gen_rom.py

Die Bilder werden nicht von Hand als Zahlen geschrieben. Das Python-Skript `scripts/gen_rom.py` zeichnet sie und erzeugt daraus `src/oled_rom_pkg.vhd`:

15. Es zeichnet das Logo in **12-facher Auflösung** (Schrift, Linien, Polygone mit der Python-Bibliothek Pillow).
16. Es **verkleinert** auf 96 x 64 Pixel; dabei entstehen weiche Kanten (Zwischenfarben).
17. Es reduziert auf **16 Farben** (Median-Cut-Verfahren) und bestimmt daraus die Palette.
18. Es schreibt jede Bildzeile als **96 Hex-Ziffern** (je Pixel eine) in die VHDL-Datei, dazu die Paletten als RGB565-Werte.

Das **HHN-Logo** ist nach der Vorlage nachgezeichnet. Wer die Original-Datei hat, legt sie als `assets/hhn_logo.png` ab und startet das Skript erneut - dann wird sie eingepasst und übernommen.

Hinweis: Nach jeder Änderung an Bildern: `python scripts/gen_rom.py` ausführen und das Vivado-Projekt neu bauen. Die Datei `oled_rom_pkg.vhd` nie von Hand ändern.

6.5 Das drehende Bosch-Symbol: Sprites

Der Bosch-Ring mit dem **Anker** (H-Form) dreht sich im Modus sw1. Eine Drehung zu berechnen (Sinus, Kosinus, Multiplikationen) wäre aufwendig und würde Kanten treppig machen. Deshalb gibt es eine einfachere Lösung: Das Skript zeichnet den Ring mit Anker **32 mal** in je 5,625 Grad Schritten (zusammen 180 Grad; ein H sieht nach einer halben Drehung wieder gleich aus) in kleine 20 x 20-Bildchen, die **Sprites**. Im FPGA wird nur das passende Bildchen ausgewählt.

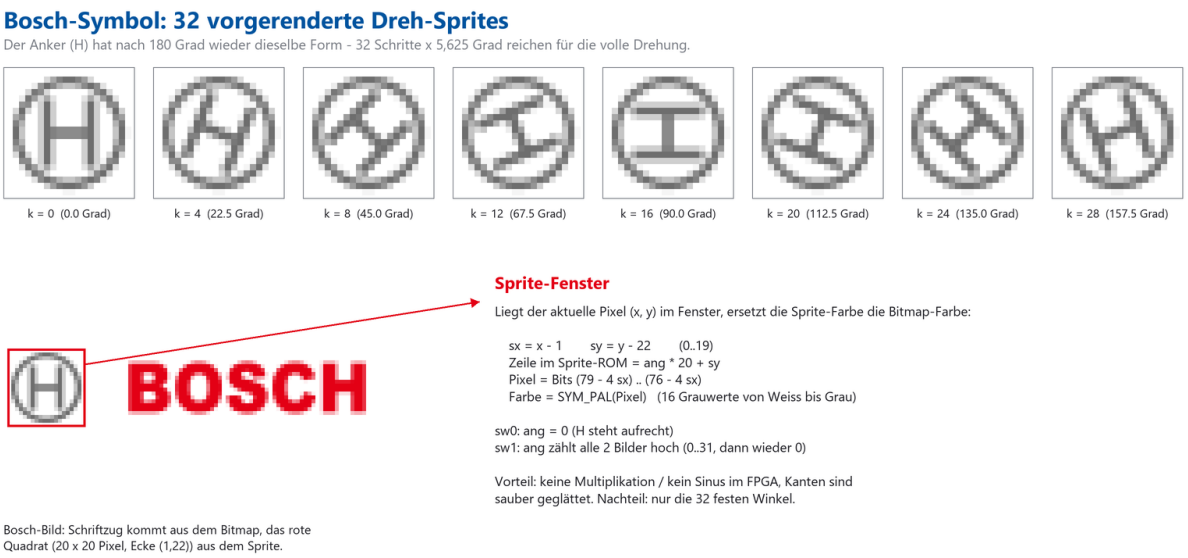


Abbildung 15: Die 32 Dreh-Sprites und das Sprite-Fenster im Bosch-Bild

- Das **Sprite-Fenster** ist ein Quadrat von 20 x 20 Pixeln mit der Ecke bei (1, 22) im Bild.
- Liegt ein Pixel (x, y) im Fenster, wird seine Farbe aus dem Sprite genommen: Zeile im Sprite-ROM = `ang * 20 + (y - 22)`, Spalte = `x - 1`.
- Der Winkelzähler ang läuft von 0 bis 31 und wird im Modus sw1 alle ROT_FRAMES = 2 Bilder um 1 erhöht; ein Umlauf von 180 Grad dauert rechnerisch 32 x 2 x 18 ms = ca. 1,2 s.
- Im Modus sw0 bleibt ang bei 0: das H steht aufrecht.

6.6 Die Pixel-Pipeline

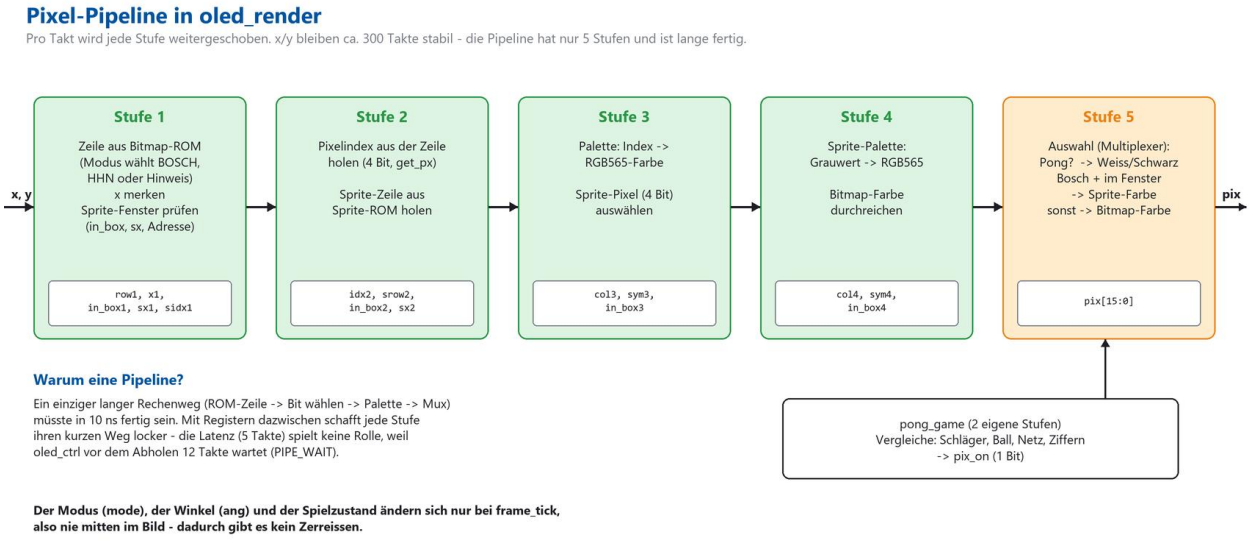


Abbildung 16: Fünf Pipeline-Stufen in oled_render

Stufe	Was wird berechnet
1	Bitmap-Zeile aus dem ROM holen (der Modus wählt Bosch/HHN/Hinweis); prüfen, ob (x, y) im Sprite-Fenster liegt und die Sprite-Adresse berechnen
2	aus der Zeile den 4-Bit-Index für Pixel x herausholen (`get_px`); Sprite-Zeile aus dem Sprite-ROM holen
3	Palette: Index -> RGB565-Farbe; Sprite-Pixel (4 Bit) herausholen
4	Sprite-Palette: Index -> Grauwert; Bitmap-Farbe durchreichen
5	Auswahl: **Pong** -> Weiß/Schwarz; **Bosch und im Fenster** -> Sprite-Farbe; sonst -> Bitmap-Farbe

Jede Stufe ist ein Registersatz. Das Ergebnis erscheint 5 Takte nach dem Anlegen von (x, y). `oled_ctrl` wartet 12 Takte, bevor es das Pixel abholt - das reicht mit Reserve, und die Pipeline kommt dadurch **ohne jede Handshake-Logik** aus. Voraussetzung: (x, y), Modus, Winkel und Spielzustand sind während dieser Zeit unverändert, und das garantiert die Aktualisierung nur bei `frame_tick`.

6.7 Die Bildschirme im Überblick

- **Bosch:** Ring + Anker links, Schriftzug BOSCH rechts, beides auf weißem Grund. Das Symbol misst nur ca. 18 Pixel, weil das Logo auf die ganze Breite des Displays (92 Pixel) skaliert ist.
- **HHN:** blaue Buchstaben "HHN" (Streben und Querbalken), darunter "HOCHSCHULE HEILBRONN" in schmaler Schrift. Die Strichgeometrie ist an der Vorlage ausgerichtet; Blauton (43, 45, 137).
- **Hinweis:** weiße Schrift "Wähle einen Schalter" auf Schwarz.
- **Pong:** Weiß auf Schwarz, siehe Kapitel 7.

7 Pong

7.1 Regeln und Bedienung

- Zwei Spieler, zwei Schläger, ein Ball. Schalter **sw3** nach oben, alle anderen unten.
- **Linker Schläger:** btn0 = hoch, btn1 = runter. **Rechter Schläger:** btn2 = hoch, btn3 = runter. Werden beide Tasten eines Schlägers gleichzeitig gedrückt, bleibt er stehen.
- Der Ball prallt oben und unten ab und wird mit jedem Schlägertreffer **schneller** (bis max. 4 Pixel pro Bild). Wo er den Schläger trifft, bestimmt den Abprallwinkel.
- Verfehlt ein Spieler den Ball, bekommt der **Gegner** einen Punkt. Nach 0,8 s Pause schlägt der **Verlierer** auf (der Ball fliegt zu ihm).
- Wer **9 Punkte** hat, gewinnt; der Stand bleibt ca. 2,7 s stehen, danach beginnt ein neues Spiel mit 0:0.
- Wird sw3 ausgeschaltet und wieder eingeschaltet, startet ebenfalls ein neues Spiel.

7.2 Das Spielfeld

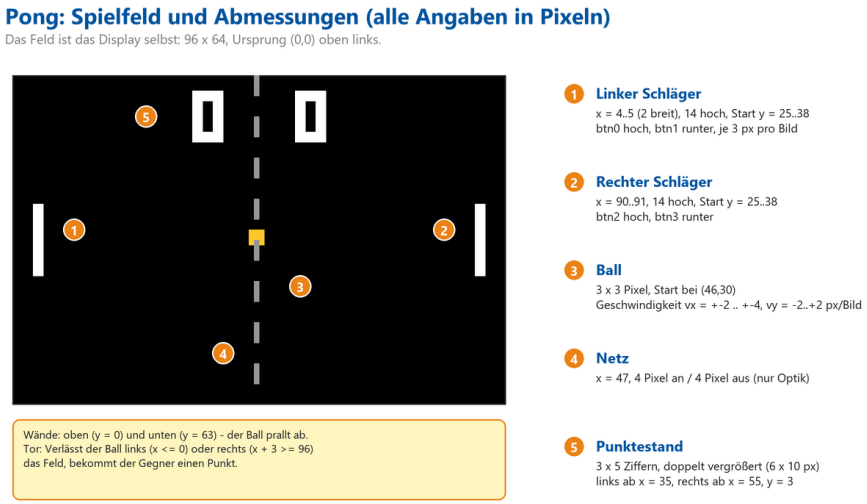


Abbildung 17: Spielfeld und Abmessungen

Konstante (pong_game.vhd)	Wert	Bedeutung
PAD_H	14	Schlägerhöhe in Pixeln
PAD_W	2	Schlägerbreite
PAD_LX, PAD_RX	4, 90	x-Position des linken bzw. rechten Schlägers
PAD_SPEED	3	Pixel pro Bild bei gedrückter Taste
BALL	3	Kantenlänge des Balls
VX_START, VX_MAX	2, 4	Start- und Höchstgeschwindigkeit in x-Richtung (Pixel pro Bild)
WIN_SCORE	9	Punkte zum Sieg
SERVE_FRAMES	45	Pause vor dem Aufschlag in Bildern (ca. 0,8 s)
OVER_FRAMES	150	Anzeigedauer nach Spielende in Bildern (ca. 2,7 s)

7.3 Der Spielzustand

Signal	Bedeutung

`pl_y`, `pr_y`	obere Kante des linken/rechten Schlägers (0 ... 50)
`bx`, `by`	obere linke Ecke des Balls
`vx`, `vy`	Geschwindigkeit in Pixeln pro Bild (vx: -4 ... +4, vy: -2 ... +2)
`score_l`, `score_r`	Punktestand (0 ... 9)
`serve_cnt`	verbleibende Aufschlag-Pause in Bildern
`over_cnt`	verbleibende Spielende-Anzeige in Bildern
`lfsr`	8-Bit-Zufallsgenerator (siehe 7.6)

Alle diese Signale ändern sich **nur bei `frame_tick`**, also etwa 55 Mal pro Sekunde. Ein "Spielschritt" ist deshalb ein Bild.

7.4 Was bei jedem Bild passiert

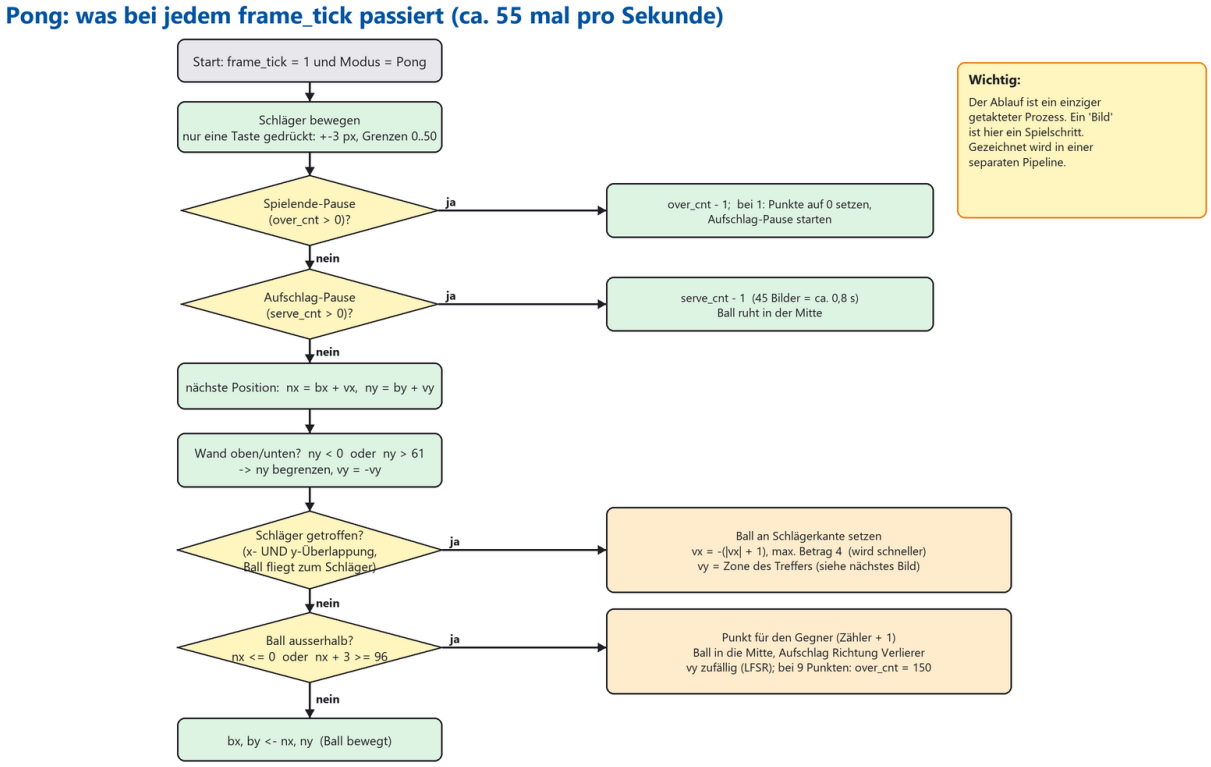


Abbildung 18: Ablauf eines Spielschritts

19. **Schläger bewegen:** Ist genau eine der beiden Tasten gedrückt, ändert sich `pl_y` um 3 Pixel, begrenzt auf 0 bis 50 (`64 - PAD_H`).
20. **Spielende?** Läuft `over_cnt`, wird nur gezählt. Bei 1 wird der Stand auf 0:0 gesetzt und der Aufschlag-Zähler gestartet.
21. **Aufschlag-Pause?** Läuft `serve_cnt`, bleibt der Ball in der Mitte.
22. **Nächste Position:** `nx = bx + vx`, `ny = by + vy`.
23. **Wand:** Ist `ny < 0` oder `ny > 61`, wird `ny` auf den Rand gesetzt und `vy` umgekehrt.
24. **Schlägertreffer?** (siehe unten)
25. **Ball draußen?** Ist `nx <= 0` oder `nx + 3 >= 96`, bekommt der Gegner einen Punkt, der Ball geht zurück in die Mitte und fliegt Richtung Verlierer; `vy` kommt aus dem Zufallsgenerator.
26. **Übernehmen:** `bx <= nx`, `by <= ny`.

```
if vx < 0 and nx <= PAD_LX + PAD_W - 1 and nx + BALL - 1 >= PAD_LX
   and ny + BALL > pl_y and ny < pl_y + PAD_H then
    -- Treffer links: x-Überlappung UND y-Überlappung
    nx := PAD_LX + PAD_W;               -- Ball an die Schlägerkante setzen
```

```
    if -vx < VX_MAX then
        vx <= -vx + 1;                  -- umkehren und 1 schneller
    else
        vx <= VX_MAX;
    end if;
    vy <= deflect(ny + 1 - pl_y);       -- Winkel nach Trefferstelle
```

Rechenbeispiel

Der Ball fliegt nach links: `bx = 12`, `by = 30`, `vx = -3`, `vy = +1`. Der linke Schläger steht bei `pl_y = 25` (belegt y = 25 bis 38).

Bild	nx	ny	Prüfung	Ergebnis
1	9	31	9 <= 5? nein	Ball fliegt weiter
2	6	32	6 <= 5? nein	Ball fliegt weiter
3	3	33	3 <= 5 und 3 + 2 >= 4: x-Überlappung. y: 33 + 3 > 25 und 33 < 39: y-Überlappung	**Treffer**: nx = 6, vx = +4, Trefferstelle 33 + 1 - 25 = 9 -> vy = +1

7.5 Der Abprallwinkel

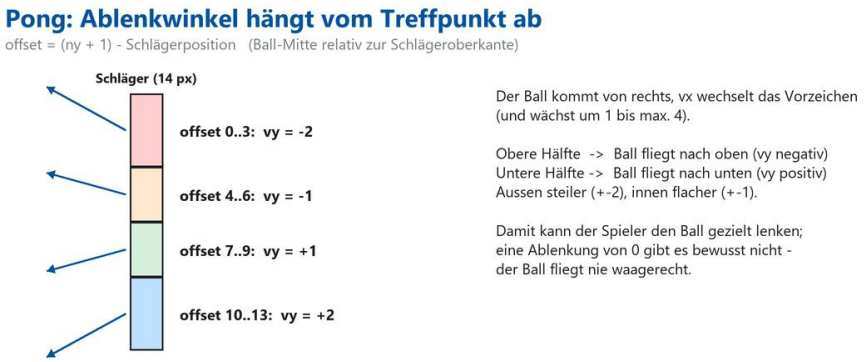


Abbildung 19: Der Treffpunkt auf dem Schläger bestimmt vy

Die Position des Balls relativ zur Schlägeroberkante (`offset`) wird in vier Zonen eingeteilt. Ein Treffer am oberen oder unteren Rand lenkt den Ball steil (+-2) ab, einer in Nähe der Mitte flach (+-1). Eine Ablenkung von 0 (exakt waagerecht) gibt es bewusst nicht.

7.6 Zufall im FPGA: das LFSR

Ein FPGA hat keinen Zufallsbefehl. Stattdessen läuft ein **LFSR** (Linear Feedback Shift Register): ein 8-Bit-Schieberegister, in das bei jedem Takt ein Bit zurückgeführt wird, das aus einigen anderen Bits per XOR gebildet wird (`lfsr(7) xor lfsr(5) xor lfsr(4) xor lfsr(3)`). Es läuft mit 100 MHz durch viele Werte. Beim Aufschlag werden die beiden untersten Bits abgelesen und auf vy = -2, -1, +1 oder +2 abgebildet. Der Zeitpunkt des Aufschlags hängt vom Spielverlauf ab, daher wirkt der Wert zufällig.

7.7 Wie Pong gezeichnet wird

Für Pong gibt es **kein Bitmap**. Für jedes (x, y) werden Vergleiche gerechnet: liegt der Punkt im linken Schläger, im rechten Schläger, im Ball, auf dem Netz (x = 47, vier Pixel an, vier aus: `y(2) = 0`) oder in einer Ziffer? Die Ziffern kommen aus einer 3 x 5-Tabelle, die doppelt vergrößert ist (6 x 10 Pixel). Ergibt irgendein Vergleich 1, ist das Pixel weiß, sonst schwarz. Das ist `pong_on`; `oled_render` macht daraus 0xFFFF oder 0x0000.

Ziffer 3 (3 x 5)	Bits
Zeile 1	111
Zeile 2	001
Zeile 3	111
Zeile 4	001
Zeile 5	111

7.8 Parameter verändern

Wunsch	Änderung
Spiel schneller	VX_START erhöhen. VX_MAX **nicht über 4** erhöhen: Der Treffer-Test fängt den Ball nur, wenn er den Schläger in einem Schritt nicht überspringt (Schläger 2 + Ball 3 Pixel breit) - bei 5 Pixeln pro Bild könnte er hindurchfliegen
Längerer Schläger	PAD_H erhöhen; Zonen in `deflect` (Schwellen 4, 7, 10) mit anpassen; Grenze 64 - PAD_H stimmt automatisch
Mehr Punkte bis zum Sieg	WIN_SCORE erhöhen; Ziffern sind nur 0 bis 9 - mehr als 9 braucht eine zweite Ziffer
Schläger flinker	PAD_SPEED erhöhen

8 Eingänge und Reset

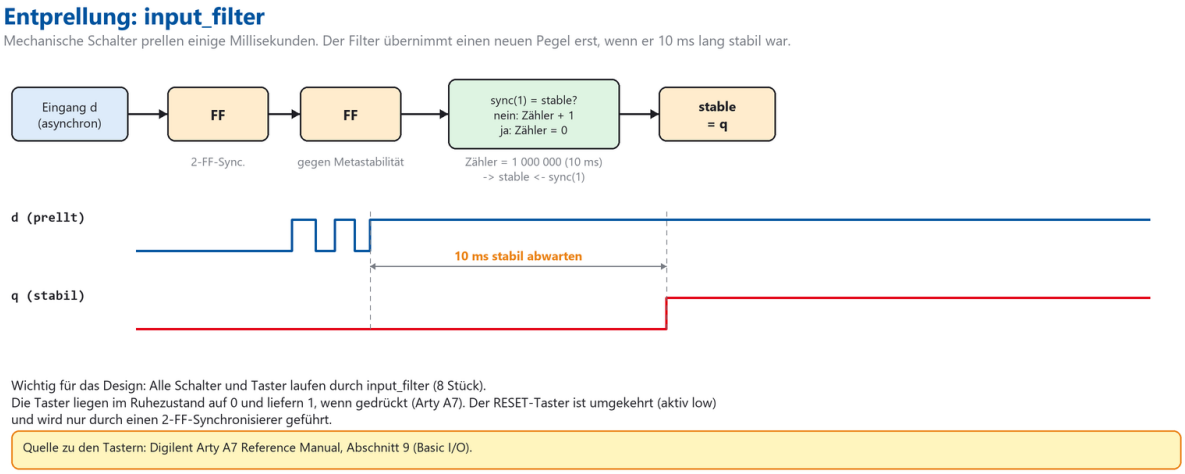


Abbildung 20: Entprellung und Einsynchronisierung

Warum Entprellen? Beim Umlegen eines Schalters federn die Kontakte für einige Millisekunden. Ohne Filter sähe der FPGA viele Flanken statt einer. **Warum einsynchronisieren?** Der Schalter ist nicht mit dem Takt synchronisiert. Ändert er sich genau an einer Taktflanke, kann ein Flip-Flop kurz einen unbestimmten Pegel ausgeben (Metastabilität). Zwei Flip-Flops hintereinander lassen diesen Zustand abklingen.

`input_filter` macht beides: Der Wert durchläuft zwei Flip-Flops. Weicht er vom ausgegebenen Wert `stable` ab, zählt ein Zähler; erst wenn der neue Pegel **1 000 000 Takte = 10 ms** lang durchgehend anliegt, wird er übernommen. Zeigt er zwischendurch wieder den alten Wert, beginnt das Zählen von vorn.

Folge für die Bedienung: Ein Schalter wirkt frühestens nach 10 ms und danach noch bis zum Ende des aktuellen Bildes (max. ca. 18 ms): also nach höchstens etwa 30 ms - unmerklich.

Reset: Der RESET-Taster liefert im Ruhezustand 1 und beim Drücken 0 [1]. Das Design invertiert ihn und führt ihn durch zwei Flip-Flops (`rst_sync`). Beim Einschalten ist `rst_sync` mit '1' vorbelegt, das Design startet also in einem definierten Zustand. Ein Druck auf RESET initialisiert das Display komplett neu (Kapitel 5.2) und setzt Pong, Drehwinkel und Modus zurück.

9 Simulation und Tests

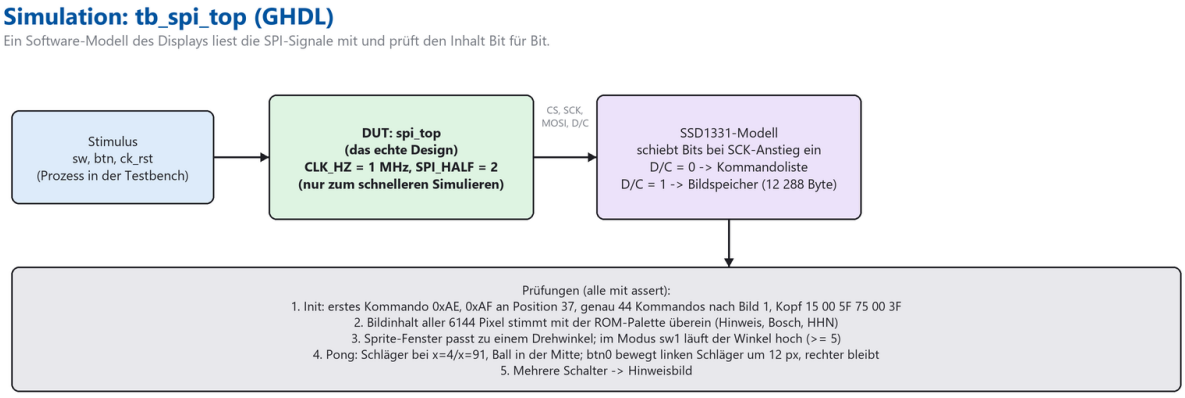


Abbildung 21: Aufbau der Simulation

Die Testbench `tb_spi_top` simuliert das Design **ohne Hardware**. Ein im VHDL geschriebenes **SSD1331-Modell** liest die SPI-Signale genauso, wie es das Display tun würde (Bits bei steigender SCK-Flanke einschieben, nach 8 Bits ein Byte; D/C = 0 -> Kommandoliste, D/C = 1 -> Bildspeicher mit 12 288 Bytes). So lässt sich das Bild, das das Display zeigen würde, im Simulator auslesen und prüfen.

Zur Beschleunigung läuft die Simulation mit `CLK_HZ = 1 MHz` (kürzere Wartezeiten) und `SPI_HALF = 2` (schnellerer SPI-Takt). Die Logik ist dieselbe.

Test	Prüfung
Initialisierung	erstes Kommando AE, AF an Position 37, genau 44 Kommandobytes nach dem ersten Bild, Fensterkommandos `15 00 5F 75 00 3F`; vor dem ersten Kommando sind PMODEN und RES = 1, vor AF ist VCCEN = 1
Kein Schalter	alle 6144 Pixel des Hinweisbildes stimmen mit dem ROM überein
sw0	Bosch-Bild stimmt außerhalb des Sprite-Fensters; im Fenster passt Winkel 0
sw1	in 20 Bildern passt das Fenster immer zu einem Sprite; der Winkel erreicht mindestens 5; der Schriftzug bleibt unverändert
sw2	HHN-Bild pixelgenau
sw3	Schläger bei x = 4 und 91, Ball in der Mitte, nach 4 Bildern mit btn0 steht der linke Schläger genau 12 Pixel höher, der rechte bewegt sich nicht
Mehrere Schalter	Hinweisbild

So startet man die Simulation:

```
powershell -File scripts/run_sim.ps1 -Ghdl <pfad>\ghdl.exe
```

Am Ende erscheint `ALLE TESTS ABGESCHLOSSEN`. Ein Lauf dauert einige Minuten, weil 20 Bilder à 6144 Pixel simuliert werden.

Achtung: Was die Simulation nicht zeigt: Timing auf dem echten Chip, die Pin-Zuordnung, das Verhalten des echten Displays (Orientierung, Farbreihenfolge) und mechanische Effekte der Taster. Das prüft man auf dem Board.

10 Bauen, Programmieren, Fehlersuche

10.1 Vivado-Projekt und Bitstream

27. In Vivado: `Tools > Run Tcl Script ...` und `scripts/create_project.tcl` wählen (oder `vivado -mode batch -source scripts/create_project.tcl`). Das Skript legt das Projekt `spi_oled` für das Bauteil `xc7a35ticsg324-1L` an, nimmt alle Dateien aus `src/`, `sim/` und `constraints/` auf und stellt die Dateien auf **VHDL 2008** ein.
28. **Generate Bitstream** ausführen (Synthese, Implementierung, Bitstream laufen nacheinander).
29. Arty per USB anschließen, **Open Hardware Manager > Auto Connect > Program Device**.
30. Die LED DONE leuchtet, wenn der FPGA konfiguriert ist [1].

10.2 Die Pin-Zuordnung (XDC-Datei)

Die Datei `constraints/arty_a7_35_oled.xdc` verbindet jeden Port des Top-Moduls mit einem FPGA-Pin und legt den Takt fest (`create_clock ... -period 10.00`). Die Pin-Namen stammen aus der Digilent-Master-XDC; für JD stimmen sie mit der Tabelle im Digilent-Handbuch überein [1].

10.3 Fehlersuche

Beobachtung	Mögliche Ursache	Maßnahme
Display bleibt schwarz	Stecker versetzt oder locker; Versorgung fehlt; Bitstream nicht geladen	Abschnitt 2.4; DONE-LED prüfen; Spannung zwischen Pin 6 und 5 messen (3,3 V)
Bild gespiegelt oder gedreht	Remap-Wert im Init	`x"72"` in `INIT_CMD` laut SSD1331-Datenblatt ändern (Remap-Bits)
Rot und Blau vertauscht	Farbreihenfolge im Remap	Remap-Wert ändern; oder im Skript die Farben tauschen
Schalter wirkt nicht	Entprellung / Moduswechsel nur am Bildende	kurz warten (< 0,05 s); LED muss mit dem Schalter mitgehen
Nichts passiert nach RESET	Reset ist aktiv low	RESET loslassen; Takt vorhanden?
Pong-Taster "verkehrt"	Zuordnung der Taster zu den Spielern	in `oled_render.vhd` (Port-Map `pong_game`) `btn(0..3)` vertauschen
Vivado meldet Fehler beim Analysieren von `oled_rom_pkg.vhd`	Datei nicht als VHDL 2008	Dateityp in Vivado auf "VHDL 2008" setzen (macht `create_project.tcl`)
Bilder geändert, aber alte Anzeige	ROM nicht neu erzeugt	`python scripts/gen_rom.py`, dann neu synthetisieren

11 Erweiterungen und Übungen

- **Übung 1 (leicht):** Ändere die Farbe des Pong-Balls im Modus sw3 (Hinweis: Farbwerte stehen in `oled_render.vhd`, Stufe 5).
- **Übung 2 (leicht):** Ändere ROT_FRAMES auf 1 oder 4 und beobachte die Drehgeschwindigkeit.
- **Übung 3 (mittel):** Eigenes Bild: ein PNG mit 96 x 64 Pixeln anlegen, in `gen_rom.py` als weiteres Bild einbinden und einem fünften Modus zuordnen.
- **Übung 4 (mittel):** Pong-Ball in der Farbe wechseln lassen, je nach Geschwindigkeit.
- **Übung 5 (anspruchsvoll):** Siegertext "GEWONNEN" anzeigen. Hinweis: Dafür bräuchte man einen kleinen Zeichensatz (Schriftrom) wie ihn die Ziffern darstellen.
- **Erweiterungsidee:** SPI-Takt auf 6,67 MHz erhöhen, um die Bildrate zu steigern (Datenblatt: min. 150 ns Periode, aktuell 160 ns - also kaum Spielraum).

12 Glossar

Begriff	Bedeutung
Arty A7-35T	FPGA-Entwicklungsboard von Digilent mit Artix-7 XC7A35T
Artix-7	FPGA-Familie von Xilinx (heute AMD)
Bitstream	Konfigurationsdatei (.bit), die die Schaltung im FPGA festlegt
CS / CS_N	Chip Select: aktiviert einen SPI-Teilnehmer (aktiv low)
Constraint (XDC)	Datei, die Ports den Pins zuordnet und den Takt beschreibt
D/C	Data/Command: Leitung, die Kommando von Pixeldaten unterscheidet
Entprellen	Filtern des Federns mechanischer Kontakte
Flip-Flop (FF)	1-Bit-Speicher, taktgesteuert
Flanke	Wechsel eines Signals von 0 nach 1 (steigend) oder 1 nach 0 (fallend)
FPGA	Field Programmable Gate Array: frei programmierbarer Logikchip
FSM	Finite State Machine: Zustandsautomat
GHDL	Freier VHDL-Simulator
JD	Vierte Pmod-Buchse des Arty A7
LFSR	Rückgekoppeltes Schieberegister, liefert Pseudozufall
LUT	Look-Up Table: kleine Wahrheitstabelle als Logikbaustein
MOSI	Master Out, Slave In: Datenleitung vom FPGA zum Display
Palette	Tabelle: Index -> Farbe
Pipeline	Hintereinander geschaltete Rechenstufen mit Registern
Pmod	Digilent-Steckstandard für kleine Erweiterungsmodule (12 Pins)
RGB565	16-Bit-Farbformat: 5 Bit Rot, 6 Bit Grün, 5 Bit Blau
ROM	Nur-Lese-Speicher; hier als Konstanten im VHDL
SCK	SPI-Takt
SPI	Serial Peripheral Interface
Sprite	Kleines Bildchen, das in ein größeres Bild eingesetzt wird
SSD1331	Display-Controller des Pmod OLEDrgb
Synthese	Umsetzen von VHDL in LUTs und Flip-Flops
Testbench	Simulationsumgebung, die ein Design anregt und prüft
VHDL	Hardware-Beschreibungssprache
Vivado	Entwicklungsumgebung von AMD/Xilinx

13 Quellen

Nr.	Quelle
[1]	Digilent: *Arty A7 Reference Manual*, https://digilent.com/reference/programmable-logic/arty-a7/reference-manual (Abschnitte: Features, Power Supplies, FPGA Configuration, Oscillators/Clocks, Basic I/O, Pmod Connectors)
[2]	Digilent: *Pmod OLEDrgb Reference Manual*, https://digilent.com/reference/pmod/pmodoledrgb/reference-manual (Features, Specifications, Interfacing with the Pmod: Power-on Sequence, Pinout Description Table)
[3]	Solomon Systech: *SSD1331 Datasheet* (von Digilent in [2] verlinkt: https://cdn-shop.adafruit.com/datasheets/SSD1331_1.2.pdf) - nur über die Verweise in [2] herangezogen
[4]	Digilent: *Arty-A7-35-Master.xdc* (Pin-Namen; im Repository als Kopie in `prj_clock/`)
[5]	Dieses Repository: `prj_spi/src/*.vhd`, `prj_spi/sim/tb_spi_top.vhd`, `prj_spi/scripts/gen_rom.py`

Hinweis: Hinweis zum Zugriff: Beim Abruf der Digilent-Seiten am 23.10.2026 lieferte der Server direkt HTTP 403. Die Inhalte von [1] und [2] wurden deshalb über die Abbilder des Internet Archive (Stand 07.11.2024) gelesen. Bitte beim Zitieren die aktuelle Fassung der Originalseiten prüfen.